

Analyse en Composantes Principales (ACP) : Théorie et Implémentation

DUT : GI-SIR-IDIA / ESTBM USMS

Pr. Abdelaziz QAFFOU

Résumé

Ce document présente l'Analyse en Composantes Principales (ACP), une méthode statistique de réduction de dimension. Nous expliquerons la théorie sous-jacente, fournirons des exemples d'implémentation en R et Python, et illustrerons la méthode avec un jeu de données concret.

Table des matières

1	Introduction à l'ACP	1
1.1	Objectifs de l'ACP	2
2	Théorie mathématique	2
2.1	Étapes principales	2
3	Tableau de données d'exemple	2
4	Implémentation en R	3
4.1	Code R complet pour l'ACP	3
4.2	Résultats de l'ACP en R	4
5	Implémentation en Python	4
5.1	Code Python complet pour l'ACP	4
6	Comparaison R/Python	7
7	Interprétation des résultats	7
8	Conclusion	7

1 Introduction à l'ACP

L'Analyse en Composantes Principales (ACP) est une technique de réduction de dimension qui transforme un ensemble de variables potentiellement corrélées en un ensemble de variables linéairement non corrélées appelées composantes principales. Cette méthode, introduite par Karl Pearson en 1901 et développée par Harold Hotelling dans les années 1930, est largement utilisée en analyse exploratoire de données et en visualisation.

1.1 Objectifs de l'ACP

- Réduire la dimensionnalité des données
- Visualiser des données multidimensionnelles
- Identifier les variables les plus importantes
- Détecter des structures sous-jacentes dans les données

2 Théorie mathématique

Soit X une matrice de données de dimensions $n \times p$ (n observations, p variables). On suppose les données centrées (moyenne nulle pour chaque variable).

2.1 Étapes principales

1. **Centrage et réduction des données** (optionnel mais recommandé) :

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

où $\bar{x}_j$ est la moyenne de la variable j et s_j son écart-type.

2. **Calcul de la matrice de covariance ou de corrélation** :

$$C = \frac{1}{n-1} Z^T Z$$

3. **Calcul des valeurs et vecteurs propres** :

$$Cv_i = \lambda_i v_i$$

où λ_i sont les valeurs propres (triées par ordre décroissant) et v_i les vecteurs propres correspondants.

4. **Calcul des composantes principales** :

$$PC_i = Zv_i$$

5. **Choix du nombre de composantes à retenir** :

- Critère de Kaiser : retenir les composantes avec valeur propre $\lambda_i > 1$
- Pourcentage de variance expliquée (généralement 70-90%)
- Scree plot (graphique des valeurs propres)

3 Tableau de données d'exemple

Considérons un jeu de données fictif représentant des mesures sur 10 individus avec 4 variables quantitatives.

TABLE 1 – Jeu de données d'exemple pour l'ACP

Individu	Taille (cm)	Poids (kg)	Âge (ans)	Revenu (k)
Ind1	175	70	25	35
Ind2	168	65	30	45
Ind3	180	80	35	50
Ind4	165	60	28	30
Ind5	170	75	40	55
Ind6	185	90	45	60
Ind7	160	55	22	25
Ind8	172	72	33	48
Ind9	178	78	38	52
Ind10	162	58	26	28

4 Implémentation en R

4.1 Code R complet pour l'ACP

```

1  # Chargement des packages nécessaires
2  library(FactoMineR)
3  library(factoextra)
4  library(ggplot2)
5
6  # Création du jeu de données
7  donnees <- data.frame(
8    Individu = paste0("Ind", 1:10),
9    Taille = c(175, 168, 180, 165, 170, 185, 160, 172, 178, 162),
10   Poids = c(70, 65, 80, 60, 75, 90, 55, 72, 78, 58),
11   Age = c(25, 30, 35, 28, 40, 45, 22, 33, 38, 26),
12   Revenu = c(35, 45, 50, 30, 55, 60, 25, 48, 52, 28)
13 )
14
15 # Affichage des données
16 print("Tableau de données :")
17 print(donnees)
18
19 # ACP avec FactoMineR (variables quantitatives seulement)
20 acp_result <- PCA(donnees[, 2:5], scale.unit = TRUE, graph = FALSE)
21
22 # Résumé des résultats
23 print("Résumé de l'ACP :")
24 print(summary(acp_result))
25
26 # Valeurs propres et pourcentage de variance
27 valeurs_propres <- acp_result$eig
28 print("Valeurs propres :")
29 print(valeurs_propres)
30
31 # Graphique des valeurs propres (Scree plot)
32 fviz_eig(acp_result, addlabels = TRUE,
33 main = "Graphique des valeurs propres (Scree plot)")
34
35 # Variables - Cercle des corrélations

```

```

36 fviz_pca_var(acp_result, col.var = "contrib",
37 gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
38 repel = TRUE,
39 title = "Cercle des corrélations")
40
41 # Individus - Projection sur le plan factoriel
42 fviz_pca_ind(acp_result, col.ind = "cos2",
43 gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
44 repel = TRUE,
45 title = "Projection des individus")
46
47 # Biplot (individus et variables)
48 fviz_pca_biplot(acp_result, repel = TRUE,
49 col.var = "#2E9FDF",
50 col.ind = "#696969",
51 title = "Biplot ACP")
52
53 # Contribution des variables aux axes
54 contrib_var <- acp_result$var$contrib
55 print("Contribution des variables aux axes :")
56 print(contrib_var)
57
58 # Coordonnées des individus sur les composantes principales
59 coord_ind <- acp_result$ind$coord
60 print("Coordonnées des individus :")
61 print(head(coord_ind))
62

```

Listing 1 – ACP en R

4.2 Résultats de l'ACP en R

Les principaux résultats de l'ACP en R incluent :

- Les valeurs propres qui indiquent la quantité de variance expliquée par chaque composante
- Le cercle des corrélations qui montre les relations entre variables
- La projection des individus sur le plan factoriel
- Le biplot qui combine individus et variables

5 Implémentation en Python

5.1 Code Python complet pour l'ACP

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.decomposition import PCA
5 from sklearn.preprocessing import StandardScaler
6 import seaborn as sns
7
8 # Création du jeu de données
9 donnees = pd.DataFrame({
10     'Individu': ['Ind1', 'Ind2', 'Ind3', 'Ind4', 'Ind5',
11     'Ind6', 'Ind7', 'Ind8', 'Ind9', 'Ind10'],

```

```

12     'Taille': [175, 168, 180, 165, 170, 185, 160, 172, 178, 162],
13     'Poids': [70, 65, 80, 60, 75, 90, 55, 72, 78, 58],
14     'Age': [25, 30, 35, 28, 40, 45, 22, 33, 38, 26],
15     'Revenu': [35, 45, 50, 30, 55, 60, 25, 48, 52, 28]
16 })
17
18 print("Tableau de données :")
19 print(donnees)
20
21 # Préparation des données (variables quantitatives seulement)
22 X = donnees[['Taille', 'Poids', 'Age', 'Revenu']]
23
24 # Standardisation des données
25 scaler = StandardScaler()
26 X_scaled = scaler.fit_transform(X)
27
28 # Application de l'ACP
29 acp = PCA()
30 X_acp = acp.fit_transform(X_scaled)
31
32 # Valeurs propres et pourcentage de variance
33 valeurs_propres = acp.explained_variance_
34 print(f"\nValeurs propres : {valeurs_propres}")
35
36 # Variance expliquée
37 variance_expliquee = acp.explained_variance_ratio_
38 print(f"\nVariance expliquée par composante : {variance_expliquee}")
39 print(f"Variance expliquée cumulative : {np.cumsum(
40 variance_expliquee)}")
41
42 # Scree plot
43 plt.figure(figsize=(10, 6))
44 plt.bar(range(1, len(variance_expliquee) + 1), variance_expliquee,
45 alpha=0.7, align='center')
46 plt.step(range(1, len(variance_expliquee) + 1),
47 np.cumsum(variance_expliquee), where='mid')
48 plt.xlabel('Composantes principales')
49 plt.ylabel('Variance expliquée')
50 plt.title('Scree plot - Variance expliquée par composante')
51 plt.grid(True, alpha=0.3)
52 plt.show()
53
54 # Cercle des corrélations
55 fig, ax = plt.subplots(figsize=(10, 8))
56 for i in range(len(acp.components_[0])):
57     ax.arrow(0, 0, acp.components_[0, i], acp.components_[1, i],
58 head_width=0.05, head_length=0.05, fc='r', ec='r')
59     ax.text(acp.components_[0, i] * 1.15, acp.components_[1, i] * 1.15,
60 X.columns[i], color='r', ha='center', va='center')
61
62 # Cercle unité
63 circle = plt.Circle((0, 0), 1, fill=False, color='blue', linestyle='
64 --', alpha=0.5)
65 ax.add_artist(circle)
66
67 # Configuration du graphique
68 plt.xlim(-1.2, 1.2)
69 plt.ylim(-1.2, 1.2)

```

```

68 plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
69 plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
70 plt.xlabel(f'Composante 1 ({variance_expliquee[0]*100:.1f}%)')
71 plt.ylabel(f'Composante 2 ({variance_expliquee[1]*100:.1f}%)')
72 plt.title('Cercle des corrélations')
73 plt.grid(True, alpha=0.3)
74 plt.axis('equal')
75 plt.show()
76
77 # Projection des individus
78 plt.figure(figsize=(10, 8))
79 scatter = plt.scatter(X_acp[:, 0], X_acp[:, 1], c=donnees['Age'],
80 cmap='viridis', s=100)
81 for i, txt in enumerate(donnees['Individu']):
82 plt.annotate(txt, (X_acp[i, 0], X_acp[i, 1]), fontsize=9)
83 plt.colorbar(scatter, label='Âge')
84 plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
85 plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
86 plt.xlabel(f'Composante 1 ({variance_expliquee[0]*100:.1f}%)')
87 plt.ylabel(f'Composante 2 ({variance_expliquee[1]*100:.1f}%)')
88 plt.title('Projection des individus sur le plan factoriel')
89 plt.grid(True, alpha=0.3)
90 plt.show()
91
92 # Biplot
93 def biplot(score, coeff, labels=None):
94 plt.figure(figsize=(10, 8))
95 xs = score[:, 0]
96 ys = score[:, 1]
97 n = coeff.shape[0]
98
99 # Individus
100 plt.scatter(xs, ys, c=donnees['Age'], cmap='viridis', s=80, alpha
101 =0.7)
102 for i, txt in enumerate(donnees['Individu']):
103 plt.annotate(txt, (xs[i], ys[i]), fontsize=9)
104
105 # Variables
106 for i in range(n):
107 plt.arrow(0, 0, coeff[i, 0]*max(xs), coeff[i, 1]*max(ys),
108 color='r', alpha=0.5, head_width=0.05)
109 if labels is None:
110 plt.text(coeff[i, 0]*max(xs)*1.15, coeff[i, 1]*max(ys)*1.15,
111 f"Var{i+1}", color='r', ha='center', va='center')
112 else:
113 plt.text(coeff[i, 0]*max(xs)*1.15, coeff[i, 1]*max(ys)*1.15,
114 labels[i], color='r', ha='center', va='center')
115
116 plt.xlabel(f'PC1 ({variance_expliquee[0]*100:.1f}%)')
117 plt.ylabel(f'PC2 ({variance_expliquee[1]*100:.1f}%)')
118 plt.title('Biplot ACP')
119 plt.grid(True, alpha=0.3)
120 plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
121 plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
122 plt.show()
123
124 biplot(X_acp, acp.components_.T, X.columns)

```

```

124 # Analyse des résultats
125 print(f"\nNombre de composantes nécessaires pour 80% de variance :")
126 cumsum = np.cumsum(variance_expliquee)
127 for i, cum in enumerate(cumsum):
128     if cum >= 0.8:
129         print(f" {i+1} composantes expliquent {cum*100:.1f}% de la variance")
130         break
131
132 print("\nMatrice des composantes (loadings) :")
133 loadings = pd.DataFrame(acp.components_.T,
134                         columns=[f'PC{i+1}' for i in range(acp.n_components_)],
135                         index=X.columns)
136 print(loadings)
137

```

Listing 2 – ACP en Python

6 Comparaison R/Python

TABLE 2 – Comparaison des approches R et Python pour l'ACP

Aspect	R (FactoMineR)	Python (scikit-learn)
Installation	<code>install.packages("FactoMineR")</code>	<code>pip install scikit-learn</code>
Standardisation	Automatique avec <code>scale.unit=TRUE</code>	Manuel avec <code>StandardScaler()</code>
Visualisation	Excellente avec <code>factoextra</code>	Manuel avec <code>matplotlib</code>
Biplot intégré	Oui	Non (à implémenter manuellement)
Sorties	Objet riche avec multiples résultats	Résultats plus basiques
Communauté	Statistique	Machine Learning

7 Interprétation des résultats

Pour notre jeu de données d'exemple, on observe généralement que :

- Les composantes 1 et 2 expliquent la majorité de la variance (souvent >80%)
- La taille et le poids sont fortement corrélés positivement
- L'âge et le revenu montrent souvent une corrélation positive
- Les individus se regroupent par profils similaires (jeunes/faible revenu vs âgés/revenu élevé)

8 Conclusion

L'ACP est une méthode puissante pour explorer des données multivariées. Elle permet de :

- Réduire la dimension des données tout en conservant l'information essentielle
- Visualiser des données multidimensionnelles
- Identifier des relations entre variables
- Détecter des outliers ou des groupes d'individus similaires

Les implémentations en R et Python offrent des approches complémentaires, avec R plus orienté vers l'analyse statistique complète et Python vers l'intégration dans des pipelines de machine learning.

Références

- Abdi, H., & Williams, L. J. (2010). Principal component analysis.
- Jolliffe, I. T. (2002). Principal component analysis (2nd ed.).
- Husson, F., Lê, S., & Pagès, J. (2017). Exploratory multivariate analysis by example using R.
- Documentation FactoMineR : <https://cran.r-project.org/web/packages/FactoMineR/>
- Documentation scikit-learn : <https://scikit-learn.org/stable/modules/generated/sklearn>